

CAP PROJECT BOOK

Production Marketplace Platform Capstone

THE PROJECT LEARNING LOOP

- Read the phase and picture the system.
- Follow one tiny teacher pattern - never a final solution.
- Build your own artifact in the supplied workspace.
- Validate with an independent control.
- Break/change one condition and recover.
- Save evidence, then open protected review only if needed.

BEGINNER-FIRST PROFESSIONAL INSTITUTE REBUILD

Build -> prove -> break -> repair -> rerun -> defend. No solved portfolio project is supplied.

This book is your teacher

Project promise

This is a portfolio build, not a solved tutorial. The book teaches the reasoning and route; your project workspace must contain the actual implementation/evidence. Integrate ingestion, quality, warehouse/dbt, orchestration, Spark/Delta decisions, Fabric boundaries, CI/CD/observability, incident recovery, clean-state reproduction, and a final technical defense.

Normal phase loop

1 why -> 2 learning -> 3 prerequisite -> 4 picture -> 5 new words -> 6 source -> 7 follow tiny pattern once -> 8 expected result -> 9 why -> 10 mistakes -> 11 recovery -> 12 independent build -> 13 validate -> 14 teach back -> 15 protected review + changed retry -> 16 evidence/mastery + route.

Protection

Never delete a bad first attempt. Review opens only after the matching attempt is saved. If weak: saved attempt -> targeted repair -> changed micro-proof -> validation -> reassessment. The starter remains incomplete by design.

Runtime truth

Capstone design can be statically reviewed here; any live Fabric/GitHub Actions/Terraform/cloud claim requires real runtime evidence.

Contents / Index

Orientation / module map / tools / routes	4-5
CAP-01 Read capstone requirements and write source contracts	6
CAP-02 Design architecture and record major ADRs	9
CAP-03 Build multi-source ingestion and raw evidence	12
CAP-04 Implement critical quality, quarantine and reconciliation	15
CAP-05 Build trusted warehouse/dbt serving layer	18
CAP-06 Orchestrate dependencies, retries and bounded backfills	21
CAP-07 Use Spark/Delta where scale justifies it	24
CAP-08 Design Fabric deployment, access and secret boundaries	27
CAP-09 Add CI/CD, monitoring, alerts and runbook	30
CAP-10 Open incident pack, diagnose and recover safely	33
CAP-11 Reproduce from clean state and freeze evidence bundle	36
CAP-12 Final technical defense and Gate 7 readiness	39
Project mastery checklist	42
Revision / project quick reference	43

CAP project map

Business/domain

Multi-source marketplace orders, sellers, and fulfillment

Integrate ingestion, quality, warehouse/dbt, orchestration, Spark/Delta decisions, Fabric boundaries, CI/CD/observability, incident recovery, clean-state reproduction, and a final technical defense.

Supplied sources

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

First-use tools

- Python + SQL/dbt reasoning
- Orchestration/DAG controls
- Spark/Delta where justified
- Fabric deployment design
- CI/CD/monitoring/runbook evidence

Prerequisite refresh

M15-M17 plus the completed P01-P03 portfolio projects

CREATE vs FILL + evidence route

Workspace rule

Work inside a learner copy/branch of **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM**. Source, STARTER, TEMPLATE, and SEALED failure/incident folders are course assets. Your implementation, diagrams, notes, logs, and evidence are learner-owned additions.

CREATE vs FILL

CREATE = make your own file/code/diagram from a blank file or incomplete starter. FILL = copy a supplied blank template, then fill your copy. Never edit protected controls or rewrite attempt-01 after seeing Review.

Per-phase evidence

Use **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/<PHASE_ID>/attempt.json**. Copy ``attempt_template.json`` to ``attempt.json`` only when beginning a genuine attempt. Run ``python 04_PHASE_EVIDENCE/check_phase_evidence.py <M18_ROOT> <PHASE_ID>``. PASS means structure is reviewable - not technical mastery.

Protected review + failure pack

Open only the matching protected Review page after the attempt is saved. Open a SEALED failure/incident pack only at its named phase. Original attempts and first-failure evidence remain append-only.

No percentage shortcut

Move on only when the phase artifact is reproducible, validation is independent, changed/failure test is complete, runtime truth is honest, and no critical failure remains unresolved.

CAP-01 Read capstone requirements and write source contracts

1. Why this matters

Code can be perfectly written and still solve the wrong problem. Project work starts by freezing who needs the output, what one record means, how fresh/correct it must be, and what failure would hurt the business.

2. What you are learning

Independently complete read capstone requirements and write source contracts and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

Project orientation complete. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Imagine a contractor asked to build a bridge without being told whether it carries bicycles or trucks. Requirements are the load limits before you choose the concrete.

5. New words before we use them

business question - The decision the data product must support.

assumption - A condition treated as true until validated.

invariant - A result/property that should remain true when the system is correct.

acceptance evidence - Observable proof that the requirement is met.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M17 architecture/system design. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: define "daily trusted order value by region" with user, freshness, output grain, one unacceptable failure, and one measurable invariant before naming a tool.

8. Expected result

You have a learner-owned **source_contracts.md** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Starting with Airflow/dbt/Spark because they look impressive.
- Writing vague goals such as "build a pipeline".
- Hiding assumptions instead of recording them.

11. Stuck / recovery help

Rewrite the goal as: user + decision + source + output grain + freshness + failure consequence + evidence. If one field is unknown, mark it as an assumption rather than inventing certainty.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: source_contracts.md. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Define expected invariant before execution, run task, compare actual vs expected, and save one independent control check.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-01; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 3. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Add/remove/change one field according to supplied compatibility rule; quarantine/fail safely rather than silently corrupt.** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-01-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-01/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-02.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-02 Design architecture and record major ADRs

1. Why this matters

Architecture turns requirements into boundaries: where data lands, where trust is established, where state lives, where failures stop, and how evidence is reproduced.

2. What you are learning

Independently complete design architecture and record major adrs and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-01 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Draw a railway map, not a photograph of every bolt. Stations are persisted boundaries; tracks are flows; switches are decisions; emergency exits are recovery paths.

5. New words before we use them

boundary - A place where responsibility/state changes.

ADR - Short decision record: context, choice, alternatives, trade-off, revisit trigger.

failure domain - Area one failure can damage together.

serving layer - Trusted output intentionally exposed to consumers.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M17 architecture/system design. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: Source -> raw/replayable -> validate/quarantine -> trusted -> serve. Mark state/checkpoint, one failure boundary, and one recovery path.

8. Expected result

You have a learner-owned **architecture.md/diagram + ADRs** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Drawing products without responsibilities.
- No replay path from raw evidence.
- No rejected alternative or revisit trigger.

11. Stuck / recovery help

Trace one source from arrival to serving. At each arrow ask: what is persisted, what can fail, what validates it, and how would I replay/recover?

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: architecture.md/diagram + ADRs. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Trace every source to trusted/serving output; identify one failure boundary and replay/recovery path.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-02; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 4. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK CAP-02: one changed-input or broken-assumption test must be run and documented before mastery.** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-02-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-02/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-03.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-03 Build multi-source ingestion and raw evidence

1. Why this matters

Ingestion is the custody chain for data. If arrival evidence is incomplete or partial success is hidden, every later transformation inherits an unprovable starting point.

2. What you are learning

Independently complete build multi-source ingestion and raw evidence and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-02 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Treat raw landing like a parcel-receipt desk: preserve what arrived, from where, when, with which identity/hash, and whether the full shipment arrived.

5. New words before we use them

raw landing - Replayable copy of received source bytes/events plus metadata.

run metadata - Run id, source identity, time, status, counts/hashes.

partial failure - Only part of the required source set arrived or succeeded.

quarantine - Preserved rejected data plus reason, not silent deletion.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M08 files/APIs + M09 ETL/quality. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: copy one file/page to a run folder, record source path + received time + hash + row count; deliberately mark an incomplete second page as PARTIAL rather than SUCCESS.

8. Expected result

You have a learner-owned **raw/landing evidence + run_metadata.json** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Overwriting raw input.
- Calling a paginated API successful after only one page.
- Dropping invalid rows without reason/evidence.

11. Stuck / recovery help

Separate receive/land from validate/publish. First prove exactly what arrived. Only then decide whether records are accepted, quarantined, retried, or blocked.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: raw/landing evidence + run_metadata.json. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Source rows/events reconcile to raw + quarantine + deliberate duplicate handling; raw evidence remains replayable.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-03; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 5. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK CAP-03: one changed-input or broken-assumption test must be run and documented before mastery..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-03-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-03/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-04.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-04 Implement critical quality, quarantine and reconciliation

1. Why this matters

A pipeline that runs can still publish wrong data. Quality gates turn business invariants into checks that can stop unsafe output and explain rejected records.

2. What you are learning

Independently complete implement critical quality, quarantine and reconciliation and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-03 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Imagine airport security: every passenger is accounted for as cleared, held for inspection, or explicitly rejected. Nobody silently disappears between counters.

5. New words before we use them

blocking check - Failure prevents publish.

warning check - Recorded issue that does not necessarily block.

reconciliation - Independent accounting from source to accepted/quarantine/dedup/serving.

control query - Second calculation/path used to catch plausible-but-wrong output.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M09 data quality + M16 DataOps. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: require non-null business id, valid amount, and known relationship; reconcile input = accepted + quarantined + deliberate duplicate handling.

8. Expected result

You have a learner-owned **tests/ + reconciliation report + quarantine evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Only testing happy-path row counts.
- Using DISTINCT to hide duplication.
- Letting tests exist without a publish consequence.

11. Stuck / recovery help

Write the invariant in plain English, define failure action, run it on a known good and known bad example, then reconcile independently.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: tests/ + reconciliation report + quarantine evidence. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Define expected invariant before execution, run task, compare actual vs expected, and save one independent control check.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-04; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 6. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK CAP-04: one changed-input or broken-assumption test must be run and documented before mastery.** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-04-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-04/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-05.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-05 Build trusted warehouse/dbt serving layer

1. Why this matters

Trusted transformations are about grain, keys, lineage, and reconciliation - not clever SQL. One unsafe join can produce believable but inflated business metrics.

2. What you are learning

Independently complete build trusted warehouse/dbt serving layer and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-04 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Imagine assembling Lego kits. Each table has a unit/grain. Joining without checking one-to-many relationships is like duplicating entire bags of pieces without noticing.

5. New words before we use them

staging - Thin typed/renamed source-aligned layer.

fact grain - What one fact row represents.

dimension key - Stable lookup/entity key used to join safely.

fan-out - Unexpected row multiplication after a join.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M07 modeling + M09 transformations. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: state left/right grain, predict join cardinality, join, compare row count/key uniqueness, then reconcile one measure to a separate query.

8. Expected result

You have a learner-owned **models/SQL/dbt + lineage + row-count controls** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Aggregating too early in staging.
- Using DISTINCT to hide fan-out.
- Building marts without documenting lineage.

11. Stuck / recovery help

Run each model/stage alone. State one-row meaning. Check keys and row counts before the join, after the join, and against an independent control.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: models/SQL/dbt + lineage + row-count controls. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

State output grain; test key/cardinality; reconcile critical measures to an independent control query.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-05; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 7. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK CAP-05: one changed-input or broken-assumption test must be run and documented before mastery..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-05-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-05/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-06.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-06 Orchestrate dependencies, retries and bounded backfills

1. Why this matters

Automation is useful only when dependencies, retries, side effects, and backfills are safe. A green scheduler UI can still publish partial or duplicated state.

2. What you are learning

Independently complete orchestrate dependencies, retries and bounded backfills and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-05 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Think of a relay race with gates. A runner cannot start because the previous runner merely entered the track; the required handoff/evidence must be complete.

5. New words before we use them

dependency - Required predecessor state/evidence.

retry - Repeat after failure under safe side-effect rules.

bounded backfill - Historical repair limited to an explicit range.

false success - Workflow reports success although required downstream truth is incomplete.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M10 Airflow + M16 production controls. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: ingest -> quality -> transform -> publish -> monitor. Force quality to fail and prove publish never advances. Then fix only the failed step and rerun bounded work.

8. Expected result

You have a learner-owned **cap_phase_artifact + evidence note** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Retries around non-idempotent side effects.
- Downstream tasks running after partial upstream success.
- Backfilling all history when only one window is damaged.

11. Stuck / recovery help

For each task write input, output, success condition, retry safety, and downstream dependency. Then inject one upstream failure before trusting automation.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: cap_phase_artifact + evidence note. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Define expected invariant before execution, run task, compare actual vs expected, and save one independent control check.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-06; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 8. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK CAP-06: one changed-input or broken-assumption test must be run and documented before mastery..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-06-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-06/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-07.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-07 Use Spark/Delta where scale justifies it

1. Why this matters

Distributed processing adds network/shuffle/partition costs. It should be used because data volume/shape needs it, not because Spark looks advanced on a portfolio.

2. What you are learning

Independently complete use spark/delta where scale justifies it and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-06 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

A group project gets slower if everyone constantly swaps papers. Partitions are work piles; shuffle is swapping piles; skew is one person receiving most of the work.

5. New words before we use them

partition - Chunk of distributed data/work.

shuffle - Redistribution across workers, often expensive.

skew - Uneven key/data distribution causing stragglers.

explain plan - Execution plan showing how the engine intends to run work.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M12 Spark + M13 Delta/lakehouse. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: compute a small trusted result locally, then perform equivalent Spark logic; compare correctness first, inspect plan/partitions second, and only then discuss optimization.

8. Expected result

You have a learner-owned **Spark job/notebook + explain plan + partition/performance evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Partitioning by high-cardinality unique id.
- Optimizing before proving equivalent results.
- Claiming Spark execution when only code was read.

11. Stuck / recovery help

Reduce to a small trusted subset, prove exact result equivalence, inspect partition counts/key distribution/plan, then change one performance variable at a time.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: Spark job/notebook + explain plan + partition/performance evidence. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Compare output to small trusted control result; inspect plan/partitions; justify performance choice without changing correctness.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-07; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 9. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Supply skewed/unbalanced input and require correctness-preserving diagnosis; optimization is secondary to invariant safety.** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-07-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-07/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-08.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-08 Design Fabric deployment, access and secret boundaries

1. Why this matters

Cloud/Fabric designs must show service, identity, access, connection, secret, environment, and deployment boundaries. A diagram is not proof of a tenant run.

2. What you are learning

Independently complete design fabric deployment, access and secret boundaries and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-07 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Treat the cloud design like a building access plan: rooms are services/workspaces, badges are identities, doors are permissions, and secrets are locked credentials - never sticky notes on the wall.

5. New words before we use them

least privilege - Only permissions needed for the task.

service identity - Non-human identity used by workload.

environment boundary - Separation such as dev/test/prod.

runtime truth - What was actually executed vs only designed/simulated.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M15 Azure/Fabric + M16 security/delivery. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: map source -> OneLake/lakehouse/warehouse/pipeline/monitoring, then mark identities, connections, secret location, dev/test/prod boundary, and what cannot be proven without tenant access.

8. Expected result

You have a learner-owned **Fabric deployment design + connection/access/secret boundary notes** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Putting secrets in config/README.
- Using broad admin access as a shortcut.
- Calling a static design a Fabric execution.

11. Stuck / recovery help

Separate design evidence from runtime evidence. For every connection ask: which identity, which permission, where secret/token lives, and how access is verified/revoked?

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: Fabric deployment design + connection/access/secret boundary notes. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Map identities/connections/access/secret boundaries; prove no plaintext secret or broad unsafe access is required.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-08; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 10. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK CAP-08: one changed-input or broken-assumption test must be run and documented before mastery..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-08-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-08/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-09.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-09 Add CI/CD, monitoring, alerts and runbook

1. Why this matters

Production readiness means a change is tested, promotable, observable, reversible, and supportable after the author is asleep.

2. What you are learning

Independently complete add ci/cd, monitoring, alerts and runbook and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-08 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Imagine shipping a mobile app update: you need tests before release, a controlled rollout, monitoring afterward, and a rollback/runbook if something breaks.

5. New words before we use them

CI/CD - Automated checks and controlled delivery/promotion.

SLA/SLO signal - Measure used to judge freshness/availability/quality target.

runbook - Actionable response steps for known operational situations.

rollback - Return to a known safe version/state.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M16 CI/CD/security/observability. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: make one safe code/config change, define pre-merge test, promotion condition, post-deploy signal, rollback trigger, and runbook step. Do not claim GitHub Actions execution without a real run log.

8. Expected result

You have a learner-owned **README.md + runbook.md + evidence_index.md** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- CI that only checks syntax.
- Monitoring without response threshold/action.
- Rollback plan that cannot restore data/state safely.

11. Stuck / recovery help

Trace change -> test -> approve -> deploy -> observe -> rollback. If any arrow has no evidence or owner, production readiness is incomplete.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: README.md + runbook.md + evidence_index.md. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Define expected invariant before execution, run task, compare actual vs expected, and save one independent control check.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-09; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 11. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK CAP-09: one changed-input or broken-assumption test must be run and documented before mastery.** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-09-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-09/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-10.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-10 Open incident pack, diagnose and recover safely

1. Why this matters

Professional evidence includes the first failure. Recovery skill is proven by diagnosis, smallest safe repair, and a fresh retry that preserves prior controls.

2. What you are learning

Independently complete open incident pack, diagnose and recover safely and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-09 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

A mechanic keeps the diagnostic code before clearing it. The original failure is evidence; erasing it makes the repair impossible to audit.

5. New words before we use them

first-failure evidence - Logs/output captured before repair.

root cause - Underlying reason the system violated an invariant.

smallest safe repair - Minimum change that resolves the cause without masking symptoms.

fresh retry - Changed/new execution after repair, not edited history.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M16 observability/incidents + M17 failure domains. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: inject a malformed page or duplicate key, capture first failure, identify the violated contract, repair the smallest component, then rerun all previously critical controls.

8. Expected result

You have a learner-owned **first-failure evidence + root_cause.md + repair + fresh retry evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Deleting the failed run.
- Changing multiple things at once.
- Calling symptom removal a root cause.

11. Stuck / recovery help

Freeze failure evidence -> state expected vs actual -> isolate boundary -> test one hypothesis -> smallest repair -> changed retry -> re-run previous critical controls.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: first-failure evidence + root_cause.md + repair + fresh retry evidence. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Save first failing evidence before repair; identify root cause; apply smallest safe repair; fresh retry must pass all prior controls.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-10; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 12. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **MANDATORY SUPPLIED FAILURE: capture first failure, diagnose, repair, fresh retry; do not overwrite original evidence..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-10-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-10/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-11.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-11 Reproduce from clean state and freeze evidence bundle

1. Why this matters

A portfolio project becomes interview evidence only when you can rebuild/defend it from clean state and explain why the design is trustworthy under changed constraints.

2. What you are learning

Independently complete reproduce from clean state and freeze evidence bundle and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-10 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Think of a driving test, not a photo of a parked car. The reviewer needs to see you start from a clean state, navigate expected problems, validate arrival, and explain your choices.

5. New words before we use them

clean-state reproduction - Rebuild from documented prerequisites rather than hidden local leftovers.

defense - Concise explanation of requirements, design, validation, failures, trade-offs, and limits.

evidence bundle - Frozen set of artifacts/logs/checks supporting claims.

revisit trigger - Changed condition that would cause redesign.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M17 architecture defense + all completed M18 evidence. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: clone/copy clean -> install/setup -> run -> validate -> inject/repair one failure -> show evidence index -> give a 90-second architecture/reliability defense.

8. Expected result

You have a learner-owned **state ledger/watermark + rerun/backfill evidence** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Demo works only on the original machine state.
- Reading README verbatim as defense.
- Claiming unexecuted cloud/runtime work.

11. Stuck / recovery help

Use defense order: business goal -> source/grain -> architecture -> critical controls -> failure/recovery -> runtime truth -> biggest trade-off -> revisit trigger.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: state ledger/watermark + rerun/backfill evidence. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Run same input twice and prove trusted logical result is stable; then add a changed/new record and prove only intended state changes.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-11; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 13. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **Replay the same batch/event plus one late/changed record; detect duplicate/logical-state corruption if implementation is unsafe..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-11-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-11/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **CAP-12.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP-12 Final technical defense and Gate 7 readiness

1. Why this matters

A portfolio project becomes interview evidence only when you can rebuild/defend it from clean state and explain why the design is trustworthy under changed constraints.

2. What you are learning

Independently complete final technical defense and gate 7 readiness and produce reviewable evidence that proves the result, not merely the action.

3. Prerequisite / start state

CAP-11 evidence saved and no unresolved critical failure. Open only the supplied source/starter/template files needed for this phase. Keep the matching protected Review and any sealed failure pack closed until the route explicitly unlocks it.

4. Picture it in your head

Think of a driving test, not a photo of a parked car. The reviewer needs to see you start from a clean state, navigate expected problems, validate arrival, and explain your choices.

5. New words before we use them

clean-state reproduction - Rebuild from documented prerequisites rather than hidden local leftovers.

defense - Concise explanation of requirements, design, validation, failures, trade-offs, and limits.

evidence bundle - Frozen set of artifacts/logs/checks supporting claims.

revisit trigger - Changed condition that would cause redesign.

See it once, then close the help

6. Useful video/source

Primary: supplied CAP workspace/source files (not an external answer).

- SOURCES/orders.csv
- SOURCES/api_pages/sellers_page_1.json + sellers_page_2.json
- SOURCES/fulfillment_snapshot.db
- SOURCES/capstone_requirements.json

Refresh if stuck: M17 architecture defense + all completed M18 evidence. Use it to recall a concept; return to this project and solve the supplied data yourself.

7. Follow with me once - tiny pattern, NOT your final solution

Toy only: clone/copy clean -> install/setup -> run -> validate -> inject/repair one failure -> show evidence index -> give a 90-second architecture/reliability defense.

8. Expected result

You have a learner-owned **defense_notes.md + reviewer checklist** that satisfies the phase outcome, can be located from the evidence record, and has explicit runtime truth. Exact protected control values are not revealed before a genuine attempt.

9. Why it worked

The pattern works because it makes the project claim falsifiable: the artifact has a stated purpose/invariant, there is an independent check, and a changed/failure condition can expose unsafe reasoning rather than rewarding a lucky happy path.

10. Common beginner mistakes

- Demo works only on the original machine state.
- Reading README verbatim as defense.
- Claiming unexecuted cloud/runtime work.

11. Stuck / recovery help

Use defense order: business goal -> source/grain -> architecture -> critical controls -> failure/recovery -> runtime truth -> biggest trade-off -> revisit trigger.

Do it yourself -> validate -> defend

12. Small independent task

CREATE the learner-owned artifact from the supplied sources/starter. **FILL** only a copied course template where one is explicitly supplied. **Never edit source masters, protected controls, or the original attempt.**

Build: defense_notes.md + reviewer checklist. Do not copy the tiny pattern as a final answer. Use the real CAP sources/starter and record assumptions/limitations.

13. Validate

Reviewer can locate every evidence artifact and challenge one design trade-off; learner explains limitation and recovery path without reading a script.

14. Teach it back

Without this page open, explain: what one input row/event represents; what you changed in CAP-12; what evidence proves it; one way the result could look plausible but be wrong; and what your changed/failure test taught you.

15. Protected review + changed retry

First save your genuine attempt. Then, only if review/repair is needed, open

05_PROTECTED_REVIEW_AND_RUBRICS/CAP_PROTECTED_PHASE_REVIEW.pdf page 14. Diagnose the smallest weak competency, close Review, and run this changed/failure proof: **PHASE CHECK CAP-12: one changed-input or broken-assumption test must be run and documented before mastery..** Append retry evidence; never overwrite the first attempt.

16. Evidence / mastery + exact next route

Evidence ID: **M18-CAP-12-EVIDENCE**

Save: **03_PROJECT_WORKSPACES/CAP_PRODUCTION_MARKETPLACE_PLATFORM/evidence/CAP-12/attempt.json**

Mastery: L3 independent professional; no unresolved critical failure; reproducible artifact/evidence.

Next: **M19 DE19-01.** Checker PASS only means evidence structure is complete; technical correctness remains a human/technical judgment.

CAP mastery checklist

Move on only if all are true

- Every canonical phase has a saved learner-owned artifact and phase evidence record.
- Source grain/keys/contracts are explicit and validated before trust.
- Critical quality/reconciliation controls catch at least one changed/bad input.
- Rerun/backfill/state behavior is deliberate and evidence-backed where applicable.
- First-failure evidence is preserved; repair is smallest safe change; retry is fresh.
- Documentation points to real artifacts/logs rather than unsupported claims.
- Runtime truth separates executed, not executed, and simulated/static work.
- A clean-state reproduction/defense is completed for the project exit phase.
- No numeric average/percentage hides an unresolved critical competency.

Project-specific protected controls

Exact QA control values remain in `CONTROL/PROTECTED\_CONTROLS/` and are not a substitute for your own independent control. A reviewer may use them after your attempt to diagnose whether your output is objectively wrong.

Next route

CAP complete -> M19 DE19-01. G07 is after M19, not directly after the capstone.

Revision / project quick reference

The five questions for every phase

1) What does one input row/event mean? 2) What invariant should remain true? 3) What artifact proves the change? 4) What independent control can catch a plausible error? 5) What changed/failure test proves transfer/recovery?

Evidence JSON minimum

phase_id -> artifact_paths -> validation(expected/actual/control) -> changed_or_failure_test -> runtime_truth -> explain_back -> unresolved_critical_failure -> next_action. Original attempt stays append-only.

Runtime language

Say **executed** only when you have a real command/run and output. Say **simulated/static** for design/fixtures. Say **not executed** for unavailable PostgreSQL/dbt/Airflow/Spark/Delta/Fabric/GitHub Actions runtime.

Interview compression

Problem -> source/grain -> architecture -> one critical invariant -> one failure/recovery -> one trade-off -> runtime truth. You should be able to give this in 60-90 seconds without reading.